

SCENARIO 1: Backend Python (Flask) - Gestione Profilo

Questo script gestisce l'aggiornamento della biografia di un utente e la visualizzazione di file di log interni per i moderatori.

Python

```
from flask import Flask, request, render_template_string
import os
```

```
app = Flask(__name__)
```

```
# Pilastro 9: Gestione dei Segreti (Configurazione errata)
```

```
app.config['SECRET_KEY'] = '12345_S3cur3_K3y_!@#'
```

```
DATABASE_URL = "postgresql://admin:PasswordTemporanea2024@localhost/db_utenti"
```

```
@app.route('/update_bio', methods=['POST'])
```

```
def update_bio():
```

```
    user_id = request.form.get('user_id')
```

```
    nuova_bio = request.form.get('bio')
```

```
    # Esecuzione query per aggiornare la bio
```

```
    # Immagina che 'db_cursor' sia già connesso
```

```
    query = f"UPDATE profiles SET bio = '{nuova_bio}' WHERE id = {user_id}"
```

```
    db_cursor.execute(query)
```

```
    return f"Ottimo! La bio per l'utente {user_id} è stata aggiornata."
```

```
@app.route('/view_log')
```

```
def view_log():
```

```
    filename = request.args.get('file')
```

```
    base_path = "/var/www/app/logs/"
```

```
    # Apertura del file richiesto
```

```
    with open(base_path + filename, 'r') as f:
```

```
        content = f.read()
```

```
    return render_template_string("<pre>{{ content }}</pre>", content=content)
```

Domande di analisi per te:

1. **SQL:** Nella funzione `update_bio`, come viene costruito il comando SQL? Quale pilastro sulla validazione degli input viene violato?
2. **Filesystem:** Nella funzione `view_log`, cosa succede se inserisco `../../../../etc/passwd` nel parametro `file`?
3. **Segreti:** Cosa noti nelle prime righe di configurazione del server?

SCENARIO 2: Frontend HTML & JavaScript - Dashboard Utente

Una pagina che mostra i dati dell'utente e gestisce un reindirizzamento dopo il logout.

HTML

```
<div id="welcome-message"></div>
```

```
<script>
```

```
  // Recupero il nome utente dall'URL (es: dashboard.html?user=Mario)
```

```
  const urlParams = new URLSearchParams(window.location.search);
```

```
  const username = urlParams.get('user');
```

```
  // Visualizzazione del messaggio di benvenuto
```

```
  if (username) {
```

```
    document.getElementById('welcome-message').innerHTML = "<h1>Benvenuto, " +  
username + "!</h1>";
```

```
  }
```

```
  // Funzione di logout con redirect dinamico
```

```
  function logout() {
```

```
    const redirectTo = urlParams.get('next');
```

```
    if (redirectTo) {
```

```
      window.location.href = redirectTo;
```

```
    } else {
```

```
      window.location.href = "/login.php";
```

```
    }
```

```
  }
```

```
</script>
```

```
<button onclick="logout()">Esci dal sistema</button>
```

Domande di analisi per te:

1. **JavaScript:** Analizza l'uso di `innerHTML`. Se un attaccante inviasse un link come `?user=<img src=x onerror=alert(1)>`, cosa accadrebbe? Quale pilastro sulla sanificazione dell'output è violato?
2. **Logic:** Nella funzione `logout()`, il sistema si fida ciecamente del parametro `next`. È possibile dirottare l'utente su un sito di phishing?

SCENARIO 3: Database SQL - Report Amministrativi

Un esempio di procedura o query complessa utilizzata per generare report sui pagamenti.

SQL

```
-- Query per estrarre i pagamenti di un cliente specifico  
-- Ipotizziamo che questa stringa sia generata da un applicativo esterno
```

```
SELECT id, data_pagamento, importo, stato  
FROM transazioni  
WHERE cliente_id = 105  
AND stato = 'COMPLETATO'  
ORDER BY data_pagamento DESC;
```

```
-- ATTENZIONE: Lo sviluppatore ha creato una funzione di "manutenzione" rapida  
-- che accetta comandi diretti per pulire i log vecchi  
DROP TABLE IF EXISTS logs_temporanei;
```

Domande di analisi per te:

1. Se il valore **105** venisse sostituito da **105; SHUTDOWN;**, cosa accadrebbe al database?
2. **Pilastro 2 (Minimo Privilegio):** Se l'applicazione web usa l'utente **root** o **db_admin** per eseguire semplici **SELECT**, quale rischio corriamo in caso di compromissione?

SCENARIO 4: JavaScript (Node.js) - API di Integrazione

Questa API riceve dati da un partner esterno per aggiornare lo stato delle spedizioni.

JavaScript

```
const express = require('express');  
const app = express();  
app.use(express.json());
```

```
app.post('/api/shipment/update', (req, res) => {  
  // Riceviamo l'oggetto della spedizione  
  const shipmentUpdate = req.body;
```

```
  // Aggiornamento massivo sul database (NoSQL o Relazionale)  
  // Esempio: shipmentUpdate = { "id": 5, "status": "Consegnato", "is_paid": true }
```

```
  db.collection('shipments').updateOne(  
    { _id: shipmentUpdate.id },  
    { $set: shipmentUpdate } // Pericoloso: sovrascrive tutto ciò che viene inviato  
  );
```

```
  res.status(200).send("Aggiornamento ricevuto");  
});
```

```
// Gestione errori generica
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send("Errore Interno: " + err.message + " in " + err.stack);
});
```

Domande di analisi per te:

1. **Logic:** Se l'utente aggiunge un campo non previsto (es: `"customer_debt": 0`) nell'oggetto JSON, il server lo accetta? (Pilastro 1: Validazione).
 2. **Error Handling:** Osserva l'ultima funzione. Quali informazioni stiamo regalando a un potenziale attaccante in caso di crash? (Pilastro 7: Gestione Errori).
-

Consigli per l'Esame:

Per ogni porzione di codice, prova a identificare:

- **L'Asset:** (Esempio: il database degli utenti, i file di sistema, la sessione del browser).
- **Il Pilastro Violato:** (Esempio: Pilastro 1 - Validazione Input, Pilastro 3 - Sanificazione Output).